BDA Week 2-3 — Distributed Association Rule Mining

Module 2 of Phase-1 Final Prep · Apriori, PCY, SON, FP-Growth

Module 2 of Phase-1 Final Prep · Apriori, PCY, SON, FP-Growth

Topics covered in this PDF:

- Web pages a user visits in a session — find sets of pages browsed together
- Words in a document — find phrases that co-occur unusually often
- Genes expressed in a tissue sample — find regulatory modules
- Movies rated by a user — feed a collaborative-filter recommender
- DNS lookups in a 1-second window — flag possible botnets
- Symptoms in a patient record — surface comorbidity patterns

CS-404 Big Data Analytics · Dr. Syed Imran Ali · Final ~ 2026-05-16

Prepared as a comprehensive professor-grade study tutor.

Contents

1. §1 — Beginning Key Notes (Study Compass)
2. §2 — The Market-Basket Model
3. §3 — Frequent Itemsets
4. §4 — Association Rules
5. §5 — The Apriori Algorithm
6. §6 — Computational Bottleneck: The Tyranny of Counting Pairs
7. §7 — PCY (Park-Chen-Yu) Algorithm
8. §8 — Multistage and Multihash Variants
9. §9 — SON Algorithm (Savasere, Omiecinski, Navathe)
10. §10 — Random Sampling and Toivonen's Algorithm
11. §11 — FP-Growth (Frequent-Pattern Growth)
12. §12 — Eight Worked Numerical Examples
13. §13 — Practice Questions (15)
14. §14 — Full Worked Answers
15. §15 — Ending Key Notes (Revision Cards)
16. §16 — Formula & Algorithm Reference

Why this PDF is mission-critical.

The midterm decoded the examiner's pattern: he loves multi-step numerical traces — Apriori candidate generation, support counting, PCY hash table updates, and association-rule generation with confidence pruning. If you walk into the hall having mechanically rehearsed the eight worked examples in §12, you have **30–40 marks locked in** before reading a single new question. The theory in §1–§11 supports the numerics; the numerics are the marks.

§1 — Beginning Key Notes (Study Compass)

These are the ten load-bearing ideas. Every numerical problem on this topic reduces to applying one of them. Memorise them in order — the Apriori pipeline is exactly idea 4 + idea 5 + idea 6 wrapped around them.

1. **Market-basket model.** Data is a set of *baskets* (transactions). Each basket is a small subset of a large universe of *items*. Goal: find subsets of items that co-occur in many baskets.
2. **Support count $\sigma(I)$.** Number of baskets that contain itemset I. Synonyms: *absolute support*, *frequency*, *count*. Always an integer.
3. **Support fraction $sup(I) = \sigma(I) / |D|$.** Probability a random basket contains I. Always between 0 and 1. **Threshold s may be given as either count or fraction — read the question carefully.**
4. **Frequent itemset.** I is frequent iff $\sigma(I) \geq s$ (or $sup(I) \geq s$). Notation: L_k = set of frequent k-itemsets.
5. **Downward-closure (anti-monotone) property.** Every non-empty subset of a frequent itemset is itself frequent. Equivalently: any superset of an *infrequent* itemset is infrequent. This is the single property that makes Apriori work.
6. **Apriori join + prune.** Generate C_{k+1} from L_k by (a) **self-joining** L_k on the first $k-1$ items in lexicographic order, (b) **pruning** any candidate whose $(k-1)$-subsets are not all in L_k.
7. **Association rule $X \rightarrow Y$** where $X \cap Y = \emptyset$. Support $sup(X \cup Y)$, confidence $conf(X \rightarrow Y) = \sigma(X \cup Y) / \sigma(X)$. Confidence is **asymmetric**: $conf(X \rightarrow Y) \neq conf(Y \rightarrow X)$ in general.
8. **PCY pass-1 trick.** During the singleton scan, hash every pair (i, j) in the current basket into a bucket and increment that bucket's counter. Pass 2 only counts pairs that (i) consist of two frequent singletons AND (ii) hashed to a bucket that exceeded threshold. The pass-1 bucket bitmap prunes the dominant memory bottleneck.
9. **SON (two-pass MapReduce).** Partition the basket file. Find candidates locally in each chunk with reduced threshold $p \cdot s$ where p is the chunk fraction. Union all local candidates. Pass 2 counts these candidates over the whole file. **No false negatives**, possible false positives that pass 2 weeds out.
10. **FP-growth (no candidate generation).** Build a compact FP-tree from two DB scans, then mine conditional pattern bases recursively. Faster than Apriori on dense data.

The single biggest exam pattern.

The examiner consistently asks one of: (a) walk a 5–10 basket dataset through Apriori passes 1–3, (b) compute confidences for all rules from a frequent itemset and pick those above a threshold, (c) fill a PCY hash table and identify frequent buckets. Master those three drills and 60–70% of the marks on this topic are guaranteed.

§2 — The Market-Basket Model

Imagine Walmart's checkout logs for a single day. Each shopper's bag is a **basket** — an unordered set of products bought together. Across millions of baskets, certain products keep appearing together: *diapers and beer*, *milk and bread*, *shampoo and conditioner*. These co-occurrence patterns drive shelf placement, cross-marketing, recommendation engines, and fraud detection.

Definitions used throughout the rest of this document.

Symbol	Meaning		
I	Universe of items (e.g. all SKUs in the store). $	\mathcal{I}	= M$.
Basket *T*	Subset of *I*. A single transaction.		
Database *D*	Collection of baskets \$T_1$, T_2, *ldots,* T_N\. $	D	= N$.
Itemset *I*	Any subset of *I*. A ***k*-itemset** has $	I	= k$.
σ(I)	Support count: $	\{T \in D : I \subseteq T\}	$.
sup(I)	Support fraction: *σ(I) / N*.		
s	Minimum-support threshold (count or fraction depending on convention).		

The model is deliberately simple and deliberately abstract. **Items** need not be physical products. They can be:

- **Web pages** a user visits in a session — find sets of pages browsed together.
- **Words** in a document — find phrases that co-occur unusually often.
- **Genes** expressed in a tissue sample — find regulatory modules.
- **Movies** rated by a user — feed a collaborative-filter recommender.
- **DNS lookups** in a 1-second window — flag possible botnets.
- **Symptoms** in a patient record — surface comorbidity patterns.

The breakthrough is that *any time you have unordered subsets drawn from a fixed universe* you can apply association rule mining. The "shopping basket" framing is just historical baggage from the original retail use case.

Analogy — Venn-style co-occurrence.

Think of every item as a region in a giant Venn diagram. The support count of \A, B, C\ is the size of the intersection $A \cap B \cap C$ measured across all baskets. Frequent itemsets are intersections that are large. Association rules are conditional probabilities computed from those intersection sizes.

Three properties of real-world basket data that drive algorithm design.

1. **Sparsity.** The universe is huge (M in the millions) but each basket is tiny (usually ≤ 20 items). So the basket × item matrix is mostly zeros.
2. **Long-tail distribution of items.** A few items appear in most baskets (milk, bread); most items appear in very few baskets. The threshold s filters out the long tail.
3. **Volume.** Real datasets are too big to fit in RAM. We need disk-friendly **pass-counting** algorithms that scan D a small constant number of times. This is exactly the design space Apriori, PCY, and SON live in.

---

§3 — Frequent Itemsets

Definition. An itemset I is **frequent** iff $\sigma(I) \geq s$ where s is the minimum-support threshold.

The support threshold can be expressed as a **count** ($s = 3$ means "at least 3 baskets") or as a **fraction** ($s = 0.3$ means "at least 30% of baskets"). They are convertible: count = fraction × $|D|$.

EXAM TRAP — count vs fraction.

Always check the units the question specifies. A phrase like "min-support = 2" with no further qualification almost always means count, especially when $|D|$ is small (4–10 baskets, as in midterm-style problems). A phrase like "min-support = 0.4" means fraction. Mis-reading this is the #1 unforced error on this topic.

Definition — *k*-itemset. An itemset with exactly k elements. L_k is the set of all frequent k-itemsets. The Apriori pipeline produces $L_1, L_2, L_3, ldots$ in order until $L_k = \emptyset$.

The downward-closure (anti-monotone) property

Theorem (downward closure).

If I is frequent then every non-empty subset $J \subset eq\ I$ is also frequent.

Proof sketch. If $I \subset eq\ T$ then $J \subset eq\ I \subset eq\ T$. So every basket that contains I also contains J. Therefore $\sigma(J) \geq \sigma(I) \geq s$, so J is frequent. *square*

The contrapositive is more useful in practice: **if any subset $J \subset neq\ I$ is *not* frequent, then I is not frequent either. **This is the** Apriori pruning principle** — and the entire reason we don't have to enumerate the 2^M possible itemsets.

Why this matters. Without downward closure, finding all frequent itemsets would be hopeless on real data. With it, we can grow itemsets layer by layer, pruning aggressively at each level.

Example. Suppose \Juice, Diaper, Nuts\ is frequent. Then *automatically* all of these are frequent:

- \Juice\, \Diaper\, \Nuts\
- \Juice, Diaper\, \Juice, Nuts\, \Diaper, Nuts\

Conversely, if \Juice, Nuts\ has support 1 (below threshold 2), we can immediately rule out \Juice, Diaper, Nuts\, \Juice, Milk, Nuts\, etc., without ever counting them.

The exponential-search-space argument.

Worst case the number of candidate itemsets is $2^M - 1$. With $M = 1000$ items that's $\approx 10^{301}$. Downward closure cuts this to (in practice) hundreds or thousands.

§4 — Association Rules

A **rule** $X \to Y$ states: "baskets that contain X tend to contain Y as well", where $X, Y \subseteq I$ are *disjoint* itemsets ($X \cap Y = \emptyset$, both non-empty).

Three numerical quality scores.

Metric	Formula	Interpretation
Support of rule	$sup(X \to Y) = \sigma(X \cup Y) / N$	Fraction of all baskets where the rule is observed.
Confidence	$conf(X \to Y) = \sigma(X \cup Y) / \sigma(X)$	Conditional probability $P(Y \mid X)$. How often does Y appear *given* X appeared?
Lift	$lift(X \to Y) = conf(X \to Y) / sup(Y) = \sigma(X \cup Y) \cdot N / (\sigma(X) \cdot \sigma(Y))$	How much more often Y appears with X than under independence. lift > 1 → positive correlation.
Conviction	$conv(X \to Y) = (1 - sup(Y)) / (1 - conf(X \to Y))$	Sensitive to direction. ∞ when conf = 1.

EXAM TRAP — confidence is asymmetric.

$conf(X \to Y) \neq conf(Y \to X)$ in general. Example: every Ferrari owner owns a car, so $conf(Ferrari \to Car) = 100\%$. But not every car owner owns a Ferrari, so $conf(Car \to Ferrari) \approx 0.001\%$. Lift is symmetric; confidence is not.

The rule-mining pipeline

```
Step 1. Find all frequent itemsets in D (Apriori / PCY / FP-growth).
Step 2. For each frequent itemset L of size >= 2:
For each non-empty proper subset X of L:
Y = L \ X
If conf(X -> Y) >= min conf:
Output rule X -> Y
```

The total rule count from a single frequent k-itemset is $2^k - 2$ (every non-empty proper subset X defines a unique rule with $Y = L \setminus X$).

Why two-stage?

Computing rules requires knowing $\sigma(X \cup Y)$ — exactly the same support count we already computed when finding the frequent itemsets. Stage 2 is therefore *almost free*: just division.

Strong rule. A rule satisfying both $sup(X \to Y) \geq s$ AND $conf(X \to Y) \geq c$ (where c is the minimum-confidence threshold).

Confidence threshold and downward-closure of confidence.

Within a fixed itemset L, if $X \to Lsetminus\ X$ has confidence below c, then for any X' $supsetneq\ X$ (so the consequent $Y' = L\ setminus\ X'$ is *smaller*), we cannot conclude anything — confidence does **not** behave monotonically with the size of X. (Some sources state a subtler rule: "if $X \to Y$ has low confidence then $X' \to Y'$ where $X' \subset neq\ X$ might still…". Don't worry about edge-case theorems for the exam — just compute confidence for every candidate rule.)

§5 — The Apriori Algorithm

Apriori is the canonical "level-wise" algorithm for finding all frequent itemsets. It exploits downward closure via the **join + prune** candidate-generation rule. It makes k passes over the database where k is the size of the largest frequent itemset.

Algorithm (pseudocode, exam form)

```
L_1 = {frequent 1-itemsets} (1 pass over DB)
for k = 1, 2, 3, ... while L_k != empty:
C_{k+1} = candidates generated from(L_k) # join + prune
for each transaction T in DB:
for each candidate c in C_{k+1}:
if c subset of T: count[c] += 1
L_{k+1} = { c in C_{k+1} : count[c] >= s }
return Union of all L_k
```

Step-by-step structure of one pass

Pass $k+1$ given L_k.

1. **Join step.** Let $p, q \in L_k$. Items in L_k are *sorted lexicographically*. Form the candidate $p \cup q$ exactly when p and q agree on their first $k - 1$ items and $p_k < q_k$ (the last item of p is lexicographically less than that of q). This produces a $(k+1)$-itemset.
2. **Prune step.** For each candidate c produced by the join, check every $(k-1)$-subset of c… wait, that's wrong. Check every ***k*-subset** of c. If any of them is **not** in L_k, drop c. (Why? Downward closure: a frequent $(k+1)$-itemset must have all $binomk+1k$ of its k-subsets frequent.)
3. **Count step.** Scan D. For each basket T, for each surviving candidate c, if $c \subset eq\ T$ increment $\sigma(c)$.
4. **Filter step.** $L_{k+1} = \backslash c \in C_{k+1} : \sigma(c) \geq s\backslash$.

EXAM TRAP — the join condition.

Many students try to join *every* pair of itemsets in L_k. **WRONG.** You only join itemsets that share their first *k-1* items in sorted order. This is what keeps the candidate count manageable. The classic mistake: producing *A, B, C*\ from *A, B*\ ∪ *A, C*\ but ALSO producing *A, B, C*\ from *A, B*\ ∪ *B, C*\ (i.e. duplicates). The first-*k-1*-match rule prevents that.

EXAM TRAP — never skip the prune step.

Many students remember the join but forget the prune. Always check that **every** *k*-subset of a *(k+1)*-candidate is in L_k. Forgetting this means counting candidates that downward closure already rules out — burning CPU and (worse on the exam) producing wrong "frequent" itemsets if the subsequent count step is not perfectly executed.

Worked candidate generation example (preview of WE 2)

Suppose L_3 = *1,2,3*\, *1,2,4*\, *1,3,4*\, *1,3,5*\, *2,3,4*\\ (items in lex order).

Join step.

- *1,2,3*\ joins with *1,2,4*\ (share first 2 items *1,2*\, *3 < 4*) → *1,2,3,4*\. ✓
- *1,3,4*\ joins with *1,3,5*\ (share first 2 items *1,3*\, *4 < 5*) → *1,3,4,5*\. ✓
- No other pairs share their first 2 items.

So C_4 = *1,2,3,4*\, *1,3,4,5*\\.

Prune step.

- *1,2,3,4*\ — its 3-subsets are *1,2,3*\, *1,2,4*\, *1,3,4*\, *2,3,4*\. All are in L_3. ✓ KEEP.
- *1,3,4,5*\ — its 3-subsets are *1,3,4*\, *1,3,5*\, *1,4,5*\, *3,4,5*\. *1,4,5*\ is **not** in L_3 (and neither is *3,4,5*\). ✗ DROP.

After the prune, C_4 = *1,2,3,4*\\. We then scan the database to count *σ(\1,2,3,4\)* and decide whether it enters L_4.

Why this is brilliant.

Without the prune, we would have wasted a database scan counting *1,3,4,5*\. The prune was a *O(1)* memory check that saved a *O(N)* disk scan. Multiplied across thousands of candidates, this is the difference between the algorithm running in minutes vs days.

Termination

The loop stops when L_k = ∅. The longest frequent itemset has size *k - 1*. Total passes = (longest frequent itemset size) + 1.

---

§6 — Computational Bottleneck: The Tyranny of Counting Pairs

Apriori's time and memory are dominated by **pass 2** — counting pairs.

Why? Three reasons:

1. **The number of candidate pairs is huge.** If there are $|L_1| = m$ frequent singletons, $|C_2| = binomm2 = m(m-1)/2$ — quadratic in m. With $m = 10^4$ frequent items, $|C_2| \approx 5 \times 10^7$ pairs.
2. **Most pairs have count 0 or 1 — but you have to count them all to know that.** Apriori doesn't yet know which pairs will turn out frequent until the pass finishes.
3. **A naive pair-count hash table doesn't fit in RAM.** Storing one 4-byte counter per pair × 5×10^7 pairs = *200 MB* — already painful, and it grows quadratically. On enterprise data with $m = 10^5$ frequent items it's ≈ *20 GB*.

For passes $k \geq 3$ the candidate counts shrink rapidly because downward closure cuts more aggressively. So pass 2 is overwhelmingly the bottleneck.

Memorise this number.

Pair-counting memory $\approx 4 \cdot binom|L_1|2$ bytes. The examiner sometimes asks you to estimate this for $|L_1| = 10^4$: $4 \cdot 50{,}000{,}000 = 200$ *MB*.

The fix. Reduce the *number of candidate pairs we need to count* before pass 2 starts. Apriori already does this via downward closure (C_2 contains only pairs of frequent singletons). PCY does it *more aggressively* by using pass 1's spare memory to pre-filter.

---

§7 — PCY (Park-Chen-Yu) Algorithm

PCY's insight: during pass 1 (counting singletons), most of memory is sitting idle. The singleton counters take $O(m)$ space; we have $O(m^2)$ of RAM to play with. Use the spare space to build a **hash table of pair counts** while we're already in the basket file.

Pass 1 of PCY

For each basket *T* in *D*:

- For each item $i \in T$: increment $count[i]$ (the singleton counter — same as Apriori).
- For each pair $(i, j) \in T$ with $i < j$: compute $k = h(i, j)$ and increment $bucket[k]$.

The hash function h maps pairs to a finite number of buckets. Each bucket is a small integer counter (maybe 4 bytes). The number of buckets is chosen so the table fills the available memory after singleton counters are stored. Typically $h(i, j) = (a \cdot i + b \cdot j)\ bmod\ B$ or a similar mixing function.

At end of pass 1. We have:

1. Singleton counts $\sigma(i)$ for every item i.
2. Bucket counts $bucket[k]$ for every bucket k.

Construct the bitmap. For each bucket k, set $bitmap[k] = 1$ iff $bucket[k] \geq s$. The bitmap takes 1 bit per bucket — orders of magnitude smaller than the bucket counters themselves.

Pass 2 of PCY

A pair (i, j) is a **candidate** iff:

1. **Both** i and j are frequent singletons (from singleton counts).

2. The bucket $h(i, j)$ exceeded the support threshold (from the bitmap).

Then count only these candidates. Most pairs are excluded by condition 2 — that's the saving.

Why this works (and the trap that students fall into)

EXAM TRAP — frequent bucket ≠ frequent pair.

A *frequent bucket* (count ≥ s) means **at least one pair** that hashes to it might be frequent. It does NOT mean every pair in that bucket is frequent. Multiple pairs hash to the same bucket; they share the count. Bucket frequency is necessary but not sufficient for pair frequency. Pass 2 still has to confirm the actual count.

Formally: if pair (i, j) is frequent, then $\sigma(i, j) \geq s$, and the bucket $h(i, j)$ contains all (i, j)'s contributions (plus other pairs' contributions), so $bucket[h(i, j)] \geq \sigma(i, j) \geq s$. So the bucket *must* be frequent. Contrapositive: an *infrequent* bucket cannot contain a frequent pair, so we can prune.

The trick is that an *infrequent* bucket gives certainty (no frequent pair inside), while a *frequent* bucket gives only a heuristic (something inside *might* be frequent — pass 2 must verify).

Storage savings — concrete numbers

Suppose $m = 10^4$ frequent singletons. Apriori's C_2 has $binom10^42 = 5 \times 10^7$ pairs. PCY with $B = 10^7$ buckets, average 5 pairs per bucket, support threshold pruning $\sim 90\%$ of buckets: pass 2 only counts $\sim 5 \times 10^6$ pairs — a 10× saving in pass-2 memory and time.

The math: PCY trades pass 1 memory (the bucket counter table) for pass 2 memory (the candidate-pair counter table). It's a win when pass 2's table dominates, which is the typical regime.

Algorithm summary

```
PASS 1:
for each basket T in D:
for each item i in T: count[i] += 1
for each pair (i, j) in T with i < j:
k = h(i, j)
bucket[k] += 1

L1 = { i : count[i] >= s }
for each bucket k:
bitmap[k] = (bucket[k] >= s)

PASS 2:
candidate_counts = {}
for each basket T in D:
for each pair (i, j) in T with i < j and i, j in L1:
if bitmap[h(i, j)] == 1:
candidate_counts[(i, j)] += 1

L2 = { (i, j) : candidate_counts[(i, j)] >= s }
```

For $k \geq 3$, PCY behaves like Apriori (the bucket trick doesn't help because pair-counts are no longer the bottleneck).

§8 — Multistage and Multihash Variants

PCY's bucket-based pruning is so effective that it begs the question: can we apply it *more than once*? Two extensions:

Multistage algorithm

Use **multiple passes of hashing** to refine the candidate-pair set.

- **Pass 1.** Same as PCY pass 1 — count singletons + hash pairs into bucket table H_1.
- **Pass 2.** Re-scan D. For each pair (i, j) that survived (both singletons frequent AND H_1-bucket frequent), hash it again with a **different** hash function h_2 into a *second* bucket table H_2. Increment H_2's bucket only when the pair would already be a PCY candidate.
- **Pass 3.** A pair becomes a C_2 candidate iff: singletons frequent AND H_1 frequent AND H_2 frequent. Count these. Output L_2.

Why it works. Each hash table independently prunes about 90% of pairs. After two stages, only ~ *1\%* remain to be counted in the final pass. The cost is one extra database scan, often worth it on memory-bound machines.

Multihash algorithm

Use **multiple hash tables in pass 1** (instead of one large one).

- **Pass 1.** Maintain two smaller hash tables H_1, H_2 with **different hash functions** h_1, h_2. For every pair, increment a bucket in *each* table.
- **Pass 2.** A pair is a candidate iff singletons frequent AND $H_1[h_1(i,j)] \geq s$ AND $H_2[h_2(i,j)] \geq s$.

Why it works. Each smaller table has more collisions per bucket but pairs only survive if *both* hashes mark frequent — so genuinely frequent pairs survive, and most spurious pairs are filtered. Empirically multihash beats PCY when the available pass-1 memory is moderate (not tiny, not abundant).

Variant	Passes	Hash tables	Best when		
Apriori	k	0	Small data, simplicity		
PCY	k	1 (in pass 1)	Large $\	C_2\	$ but ample pass-1 memory
Multistage	$k+1$ or more	≥ 2 (sequential)	Pass-2 memory is the binding constraint		
Multihash	k	≥ 2 (parallel in pass 1)	Pass-1 memory moderate; want one extra scan		

§9 — SON Algorithm (Savasere, Omiecinski, Navathe)

SON is the **two-pass distributed** ARM algorithm. It is the answer to: "*How do I run association rule mining when the data is too big for a single machine?*" It is naturally MapReduce-friendly and is the algorithm the examiner is most likely to ask about under a "Distributed Apriori" framing.

Core idea

SON Lemma.

If an itemset I is frequent in the entire database D, then I must be frequent in **at least one** chunk of any partition of D.

Why? Suppose D is split into p equal chunks D_1, *ldots,* D_p. If I's count is below s in *every* chunk, then $\sigma_{chunk}(I) < s/p$ in each chunk, so total $\sigma(I) < s$ — contradicting that I is frequent. (The bound is more precise when chunks are unequal, but the principle stands.)

The contrapositive is the operational version: **if *I* is not frequent in any chunk with the local threshold, *I* cannot be frequent globally — skip it.**

Algorithm

Pass 1 (find candidates locally).

- Partition D into chunks D_1, *ldots,* D_p. Each chunk is fraction $p_i = |D_i| / |D|$ of the data; usually $p_i \approx 1/p$.
- In each chunk D_i, run a local frequent-itemset miner (Apriori, PCY, FP-growth — your choice) with **reduced threshold** $p_i \cdot s$. (If s is given as a count and chunks are equal, the local count threshold is s/p.)
- Collect every itemset that was frequent in *any* chunk — this is the global candidate set C.

Pass 2 (count globally).

- Scan all of D. For each candidate $I \in C$, count $\sigma(I)$ exactly.
- Output $L = \{I \in C : \sigma(I) \geq s\}$ — the true global frequent itemsets.

No false negatives by the SON lemma. **Possible false positives** (something local-frequent might not be global-frequent) — pass 2 weeds them out.

MapReduce flavour

Pass 1.

- *Map.* Each mapper reads one chunk D_i, runs local mining with threshold $p_i \cdot s$, emits each local frequent itemset I as $(I, 1)$.
- *Reduce.* For each unique itemset I received from any mapper, emit I. (The reduce just deduplicates.) Output: candidate set C.

Pass 2.

- *Map.* Each mapper receives the candidate set C (broadcast) and a chunk D_i. For each basket T and each candidate $I \subseteq T$, emit $(I, 1)$.
- *Reduce.* Sum up the 1's. If total $\geq s$, emit I.

Two MapReduce jobs total — pass 1 finds candidates, pass 2 verifies them.

EXAM TRAP — local threshold is reduced.

Setting the local threshold to the *global* threshold s is a common mistake. With three chunks and $s = 12$, the local threshold should be $12/3 = 4$, not 12. If you use s locally you risk false negatives (an itemset frequent globally but spread evenly across chunks could end up just below s in every chunk and get missed).

Why SON dominates the distributed setting

- **Embarrassingly parallel pass 1.** Each chunk is independent.
- **Two passes total**, regardless of the size of the longest frequent itemset. Compare with Apriori's k passes.
- **Tunable.** You can choose the local algorithm freely — often FP-growth in pass 1, exact counting in pass 2.
- **Scales linearly.** Throughput grows with chunk count.

The cost is a (potentially large) candidate set C that must be broadcast in pass 2. In practice this is usually small enough.

---

§10 — Random Sampling and Toivonen's Algorithm

Two more refinements you should *recognise* even if not derive in full.

Sampling (one-pass approximation)

Take a random sample $D' \subset D$ of fraction p. Run Apriori on D' with threshold $p \cdot s$ — looser to compensate for the smaller sample. The frequent itemsets in D' approximate those in D.

Pros. One pass on a small sample. Fast. **Cons.** Both false positives and false negatives are possible. Need a second pass to verify (then it becomes essentially SON with a single chunk).

Toivonen's algorithm

A clever sampling variant that **eliminates false negatives** (with high probability) using one pass plus a small second pass.

1. Take a random sample. Run Apriori with **lowered** threshold (e.g. $0.9 \cdot p \cdot s$) → set F of frequent itemsets *in the sample*.
2. Compute the **negative border** of F — itemsets NOT in F but all of whose subsets are.
3. One pass over full D: count every itemset in $F \cup$ negative border.
4. If no negative-border itemset is frequent in D, output F's frequent ones — DONE, no false negatives.
5. If some negative-border itemset is frequent globally, restart with a different sample.

Beautiful theory; rare on undergrad finals. Memorise the high-level idea.

---

§11 — FP-Growth (Frequent-Pattern Growth)

FP-growth abandons the join-and-count loop entirely. Instead it builds a compact tree representation of the database and mines patterns recursively. Two database scans total, no candidate generation.

FP-tree construction

Scan 1. Count singleton supports. Find L_1, sort items in L_1 by descending support — call this the **header table**.

Scan 2. For each basket:

- Drop infrequent items.
- Sort remaining items by header-table order (descending support).
- Insert the sorted basket into the FP-tree, sharing prefixes with existing branches and incrementing node counters.

The FP-tree compresses the database: shared prefixes are shared in the tree. Each node stores (item, count, link to next same-item node). The header table provides a linked list across all occurrences of each item.

Mining the FP-tree (conditional pattern bases)

For each item *a* in the header table (from least frequent upward):

1. **Conditional pattern base of *a*.** Trace every path in the tree ending at *a*; record the prefix path (excluding *a*) along with *a*'s count. This gives a small "sub-database" of patterns containing *a*.
2. **Build a conditional FP-tree** for that sub-database.
3. Recurse: mine the conditional FP-tree to find frequent itemsets ending in *a*.

Pattern emitted: *a*'s suffix concatenated with each pattern from the recursion.

Why it's fast

- No candidate generation → no $|C_2|$ explosion.
- Database is compressed in the tree → memory-friendly when many baskets share prefixes.
- Recursion explores only patterns that *actually exist* in the data.

When FP-growth wins.

Dense data with many shared prefixes. Apriori wins when data is sparse and pass 2 fits comfortably in RAM.

Limitations

- The FP-tree must fit in main memory (else: partition the database, build per-partition trees, then aggregate).
- Implementation is more complex than Apriori — students often find it harder to trace by hand.

For the exam, you should be able to (i) describe the two scans, (ii) construct an FP-tree from a small dataset (5–8 baskets), (iii) read off the conditional pattern base of one item.

§12 — Eight Worked Numerical Examples

These are exam-style problems. **Read the worked solution once, then close this document and try to reproduce it from scratch on paper.** Time yourself: 6–10 minutes per example.

Worked Example 1 — Singletons and L_1 from 5 baskets

Problem. Database has 5 baskets and items \a, b, c, d, e\. Threshold $s = 2$ (count). Find $\sigma(\cdot)$ for every singleton and identify L_1.

TID	Items
T1	a, b, c
T2	b, d
T3	a, b, c, d
T4	a, c, e
T5	b, c, e

Step 1 — count each item's appearances.

Item	Baskets containing it	σ
a	T1, T3, T4	3
b	T1, T2, T3, T5	4
c	T1, T3, T4, T5	4
d	T2, T3	2
e	T4, T5	2

Step 2 — compare against $s = 2$. All five items meet the threshold (the smallest count is 2, exactly s).

Result. L_1 = \\a\, \b\, \c\, \d\, \e\\.

> **Note.**
>
> "Frequent" means $\sigma \geq s$, *not* $\sigma > s$. The boundary case $\sigma = s$ is included.

Worked Example 2 — Candidate C_3 from L_2 (join + prune)

Problem. Given L_2 = \\1,2\, \1,3\, \1,5\, \2,3\, \2,5\\ (items in lex order). Generate C_3.

Step 1 — join. Pair every two itemsets in L_2 that share their first $k - 1 = 1$ item, and where the second item of the first is less than the second item of the second.

- \1,2\ and \1,3\ share \1\, $2 < 3$ → \1,2,3\. ✓
- \1,2\ and \1,5\ share \1\, $2 < 5$ → \1,2,5\. ✓
- \1,3\ and \1,5\ share \1\, $3 < 5$ → \1,3,5\. ✓
- \2,3\ and \2,5\ share \2\, $3 < 5$ → \2,3,5\. ✓

- No others share a first item.

Joined candidate set: \1,2,3\, \1,2,5\, \1,3,5\, \2,3,5\.

Step 2 — prune. For each candidate, check that *all* of its 2-subsets are in L_2.

Candidate	2-subsets	All in L_2?	Verdict
\1,2,3\	\1,2\, \1,3\, \2,3\	Yes	KEEP
\1,2,5\	\1,2\, \1,5\, \2,5\	Yes	KEEP
\1,3,5\	\1,3\, \1,5\, \3,5\	\3,5\ $\notin L_2$	DROP
\2,3,5\	\2,3\, \2,5\, \3,5\	\3,5\ $\notin L_2$	DROP

Result. C_3 = \\1,2,3\, \1,2,5\\.

Why the prune matters.

Without pruning, we would have proceeded to count \1,3,5\ and \2,3,5\ in pass 3 — wasted scans. The prune is a $O(|C_3| \cdot k)$ memory check that saves $O(N \cdot |C_3|)$ disk work.

Worked Example 3 — Full Apriori on 6 baskets

Problem. Items \B, M, Be, E, D\ = \Bread, Milk, Beer, Eggs, Diaper\. Threshold $s = 2$ (count). Find all frequent itemsets up to L_3.

TID	Items
T1	B, M
T2	B, D, Be, E
T3	M, D, Be, E
T4	B, M, D, Be
T5	B, M, D, E
T6	M, D

Pass 1 — singleton counts.

Item	σ	Frequent?
B (Bread)	4 (T1, T2, T4, T5)	✓
M (Milk)	5 (T1, T3, T4, T5, T6)	✓
Be (Beer)	3 (T2, T3, T4)	✓
E (Eggs)	3 (T2, T3, T5)	✓
D (Diaper)	5 (T2, T3, T4, T5, T6)	✓

L_1 = \B, M, Be, E, D\ — all five frequent. (Sort lex: B < Be < D < E < M for the join step. To keep notation light, I'll use the order **B, M, Be, E, D** as listed but conceptually treat them as ordered for joining.)

To make joins clean, use lex order **B < Be < D < E < M**.

Pass 2 — generate C_2 by joining L_1 with itself.

C_2 = all pairs from L_1 (since each singleton's first *0* items trivially match):

\B,Be\, \B,D\, \B,E\, \B,M\, \Be,D\, \Be,E\, \Be,M\, \D,E\, \D,M\, \E,M\.

(10 pairs = *binom52*.)

Count each by scanning the 6 baskets:

Pair	Baskets	σ	Frequent?
\B, Be	T2, T4	2	✓
\B, D	T2, T4, T5	3	✓
\B, E	T2, T5	2	✓
\B, M	T1, T4, T5	3	✓
\Be, D	T2, T3, T4	3	✓
\Be, E	T2, T3	2	✓
\Be, M	T3, T4	2	✓
\D, E	T2, T3, T5	3	✓
\D, M	T3, T4, T5, T6	4	✓
\E, M	T3, T5	2	✓

L_2 = *\\B,Be\, \B,D\, \B,E\, \B,M\, \Be,D\, \Be,E\, \Be,M\, \D,E\, \D,M\, \E,M* — all 10 pairs.

Pass 3 — generate C_3 by join + prune on L_2.

Join: for itemsets sharing first 1 item:

- Sharing *B*: *\B,Be\, \B,D\, \B,E\, \B,M* → join pairs *\B,Be,D\, \B,Be,E\, \B,Be,M\, \B,D,E\, \B,D,M\, \B,E,M*.
- Sharing *Be*: *\Be,D\, \Be,E\, \Be,M* → join pairs *\Be,D,E\, \Be,D,M\, \Be,E,M*.
- Sharing *D*: *\D,E\, \D,M* → join pair *\D,E,M*.

C_3 (after join, before prune): *\B,Be,D\, \B,Be,E\, \B,Be,M\, \B,D,E\, \B,D,M\, \B,E,M\, \Be,D,E\, \Be,D,M\, \Be,E,M\, \D,E,M*. That's 10 candidates.

Prune: each must have all three of its 2-subsets in L_2. Since $|L_2|$ = *10* = all pairs, every subset is in L_2. **All 10 candidates survive the prune.**

Count each on the 6 baskets:

Triple	Baskets	σ	Frequent?
\B, Be, D	T2, T4	2	✓
\B, Be, E	T2	1	✗
\B, Be, M	T4	1	✗
\B, D, E	T2, T5	2	✓
\B, D, M	T4, T5	2	✓

Triple	Baskets	σ	Frequent?
\B, E, M\	T5	1	✗
\Be, D, E\	T2, T3	2	✓
\Be, D, M\	T3, T4	2	✓
\Be, E, M\	T3	1	✗
\D, E, M\	T3, T5	2	✓

L_3 = \\B,Be,D\, \B,D,E\, \B,D,M\, \Be,D,E\, \Be,D,M\, \D,E,M\\ — 6 frequent triples.

Pass 4 (preview). C_4 candidates would be quadruples sharing first 2 items.

- \B,Be,D\ and \B,D,E\ — share \B\ but the second items differ (*Be* vs *D*). Don't share first 2 items. **No join.**

Wait — re-check the lex order: *B* < *Be* < *D* < *E* < *M*. So \B,Be,D\'s first 2 items are *B, Be*, and \B,D,E\'s are *B, D*. Don't match.

- \B,D,E\ and \B,D,M\ — share \B, D\, last items *E* < *M*. Join → \B,D,E,M\.
- \Be,D,E\ and \Be,D,M\ — share \Be, D\, last items *E* < *M*. Join → \Be,D,E,M\.

C_4 before prune: \B,D,E,M\, \Be,D,E,M\.

Prune. Check 3-subsets:

- \B,D,E,M\: 3-subsets are \B,D,E\, \B,D,M\, \B,E,M\, \D,E,M\. \B,E,M\ is **NOT** in L_3. → DROP.
- \Be,D,E,M\: 3-subsets are \Be,D,E\, \Be,D,M\, \Be,E,M\, \D,E,M\. \Be,E,M\ is **NOT** in L_3. → DROP.

C_4 after prune is empty. L_4 = ∅. Algorithm terminates.

All frequent itemsets: L_1 ∪ L_2 ∪ L_3 — 5 singletons + 10 pairs + 6 triples = 21 frequent itemsets total.

Worked Example 4 — Generate strong rules from a frequent itemset

Problem. Frequent itemset *L* = \I_1, I_2, I_5\ with *σ(L) = 2*. Other relevant supports (from a 9-basket database used in slides):

Itemset	σ
\I_1\	6
\I_2\	7
\I_5\	2
\I_1, I_2\	4
\I_1, I_5\	2
\I_2, I_5\	2
\I_1, I_2, I_5\	2

Min confidence threshold *c = 70*\%. Find all **strong rules** derived from *L*.

Method. For every non-empty proper subset X of L, form rule $X \to L \text{ setminus } X$ and compute confidence $\sigma(L) / \sigma(X)$. With $|L| = 3$, there are $2^3 - 2 = 6$ rules.

Rule	σ(numerator)	σ(X)	Confidence	Strong? (≥ 70\%)
$\{I_1, I_2\} \to \{I_5\}$	2	4	50%	✗
$\{I_1, I_5\} \to \{I_2\}$	2	2	100%	✓
$\{I_2, I_5\} \to \{I_1\}$	2	2	100%	✓
$\{I_1\} \to \{I_2, I_5\}$	2	6	33%	✗
$\{I_2\} \to \{I_1, I_5\}$	2	7	29%	✗
$\{I_5\} \to \{I_1, I_2\}$	2	2	100%	✓

Three strong rules:

- $\{I_1, I_5\} \to \{I_2\}$ (conf = 100%)
- $\{I_2, I_5\} \to \{I_1\}$ (conf = 100%)
- $\{I_5\} \to \{I_1, I_2\}$ (conf = 100%)

Pattern.

When I_5 appears it almost always appears with I_1 and I_2 — very strong association. The "anchor" is the rare item I_5. This is typical of real association rules: rare consequents/antecedents drive high confidence.

EXAM TRAP — confidence asymmetry.

Note $\{I_5\} \to \{I_1, I_2\}$ has confidence 100% but $\{I_1, I_2\} \to \{I_5\}$ has confidence 50%. Same itemset, different rules, very different confidences. **Always compute both directions if asked for "all rules from this itemset".**

Worked Example 5 — PCY hash table fill

Problem. 5 baskets, items $\{1, 2, 3, 4, 5\}$. Hash function $h(i, j) = (10\ i + j) \text{ bmod } 7$. Threshold $s = 3$ (count, applies to both items and buckets).

TID	Items
T1	1, 2, 3
T2	2, 3, 5
T3	1, 2, 5
T4	1, 3, 4
T5	2, 3, 4, 5

Step 1 — pass 1 singleton counts.

Item	Baskets	σ	Frequent?
1	T1, T3, T4	3	✓

Item	Baskets	σ	Frequent?
2	T1, T2, T3, T5	4	✓
3	T1, T2, T4, T5	4	✓
4	T4, T5	2	✗
5	T2, T3, T5	3	✓

L_1 = \1, 2, 3, 5\. Item 4 is dropped.

Step 2 — pass 1 hash buckets. For every basket, every pair *(i, j)* with *i* < *j* is hashed and bucket count incremented.

Pairs in each basket:

Basket	Pairs
T1 = \1,2,3\	*(1,2), (1,3), (2,3)*
T2 = \2,3,5\	*(2,3), (2,5), (3,5)*
T3 = \1,2,5\	*(1,2), (1,5), (2,5)*
T4 = \1,3,4\	*(1,3), (1,4), (3,4)*
T5 = \2,3,4,5\	*(2,3), (2,4), (2,5), (3,4), (3,5), (4,5)*

Hash each pair: *h(i, j) = (10 i + j) bmod 7*.

Pair	*10i + j*	*bmod 7*	Bucket
(1,2)	12	5	5
(1,3)	13	6	6
(1,4)	14	0	0
(1,5)	15	1	1
(2,3)	23	2	2
(2,4)	24	3	3
(2,5)	25	4	4
(3,4)	34	6	6
(3,5)	35	0	0
(4,5)	45	3	3

Tally pair occurrences and per-bucket totals.

Pair	Occurrences	List of TIDs
(1,2)	2	T1, T3
(1,3)	2	T1, T4
(1,4)	1	T4

Pair	Occurrences	List of TIDs
(1,5)	1	T3
(2,3)	3	T1, T2, T5
(2,4)	1	T5
(2,5)	3	T2, T3, T5
(3,4)	2	T4, T5
(3,5)	2	T2, T5
(4,5)	1	T5

Bucket counts.

Bucket	Pairs hashed there	Total count
0	(1,4): 1 + (3,5): 2	**3** ✓ frequent
1	(1,5): 1	**1** ✗
2	(2,3): 3	**3** ✓ frequent
3	(2,4): 1 + (4,5): 1	**2** ✗
4	(2,5): 3	**3** ✓ frequent
5	(1,2): 2	**2** ✗
6	(1,3): 2 + (3,4): 2	**4** ✓ frequent

Bitmap. *bitmap = (1, 0, 1, 0, 1, 0, 1)* for buckets 0..6.

Step 3 — pass 2 candidate construction. A pair is a candidate iff:

1. Both items are in L_1 = *1, 2, 3, 5*\.
2. Its bucket is frequent.

Apply both filters:

Pair	Both items in L_1?	Bucket frequent?	Candidate?
(1,2)	✓	bucket 5 = 2: ✗	NO
(1,3)	✓	bucket 6 = 4: ✓	**YES**
(1,5)	✓	bucket 1 = 1: ✗	NO
(2,3)	✓	bucket 2 = 3: ✓	**YES**
(2,5)	✓	bucket 4 = 3: ✓	**YES**
(3,5)	✓	bucket 0 = 3: ✓	**YES**

(Pairs containing item 4 are pre-filtered by step 1 since 4 is not in L_1.)

Pass 2 candidates: *1,3*\, *2,3*\, *2,5*\, *3,5*\.

Step 4 — pass 2 actual count.

Pair	σ	≥ 3?	In L_2?
(1,3)	2	✗	No
(2,3)	3	✓	YES
(2,5)	3	✓	YES
(3,5)	2	✗	No

L_2 = \\2,3\, \2,5\\.

Reading the savings.

Plain Apriori would have counted all *binom42 = 6* pairs in L_1. PCY counted only 4. Two pairs were eliminated by infrequent buckets (\1,2\ via bucket 5, \1,5\ via bucket 1). On a tiny example this is small, but the same proportion at $|L_1| = 10^4$ would save tens of millions of pair-counts.

EXAM TRAP — bucket 5 is NOT frequent.

Even though pair *(1,2)* alone has count 2, and the question's threshold is 3, the *bucket* count is 2 (only *(1,2)* hashes to it). 2 < 3, so bucket 5 is infrequent, so *(1,2)* is pruned BEFORE it would even be counted in pass 2. PCY can prune true frequent-singleton pairs if their bucket happens to fall short. That's the algorithm working as intended — *and* why pass 2 must verify with an actual count.

Worked Example 6 — Confidence and lift for \milk\ → \bread\

Problem. From a 1000-basket database: *σ(milk) = 600*, *σ(bread) = 400*, *σ(\milk, bread\) = 240*. Compute the support, confidence, and lift of the rule *milk*\ → *bread*\. Interpret.

Step 1 — support of the rule.

$$\text{sup}(\{\text{milk}\} \to \{\text{bread}\}) = \frac{\sigma(\text{milk} \cup \text{bread})}{N} = \frac{240}{1000} = 0.24 = 24\%$$

Step 2 — confidence.

$$\text{conf}(\{\text{milk}\} \to \{\text{bread}\}) = \frac{\sigma(\text{milk} \cup \text{bread})}{\sigma(\text{milk})} = \frac{240}{600} = 0.40 = 40\%$$

Step 3 — lift.

$$\text{lift} = \frac{\text{conf}(\{\text{milk}\} \to \{\text{bread}\})}{\text{sup}(\{\text{bread}\})} = \frac{0.40}{400/1000} = \frac{0.40}{0.40} = 1.00$$

Interpretation.

- **Support 24%** — the pair appears in nearly a quarter of all baskets. Substantial.
- **Confidence 40%** — given a basket has milk, there's a 40% chance it also has bread.
- **Lift 1.00** — but bread appears in 40% of *all* baskets too! So milk doesn't actually predict bread better than a random guess. The two are **statistically independent**.

Lesson.

High confidence does not imply association. Always check lift. Lift = 1 is the "no association" baseline. Lift > 1 = positive association. Lift < 1 = negative (anticorrelation).

Sanity check the other direction. *conf(\bread\ → \milk\) = 240/400 = 60\%*. Different confidence, but lift is still *0.60 / 0.60 = 1.00*. Lift is symmetric; confidence isn't.

Worked Example 7 — SON algorithm on 12 baskets, 3 chunks

Problem. 12 baskets split into 3 chunks of 4. Items *\A, B, C, D*. Global threshold *s = 4* (count). Find global frequent itemsets via SON. Local threshold per chunk = *s · p = 4 · (1/3) = 4/3 ≈ 1.33* → round up to **2**.

Chunk	TIDs	Items
1	T1: *\A,B,C* ; T2: *\A,B* ; T3: *\A,C* ; T4: *\B,C*	
2	T5: *\A,B,C* ; T6: *\A,B,D* ; T7: *\B,C,D* ; T8: *\A,C,D*	
3	T9: *\A,B,C,D* ; T10: *\B,C* ; T11: *\A,B* ; T12: *\A,C*	

Pass 1 — local mining with threshold = 2.

Chunk 1.

- Singletons: *A: 3, B: 3, C: 3*. All ≥ 2. $L_1^{(1)}$ = *\A, B, C*.
- Pairs: *\A,B\: 2* (T1, T2), *\A,C\: 2* (T1, T3), *\B,C\: 2* (T1, T4). All ≥ 2. $L_2^{(1)}$ = *\\A,B\, \A,C\, \B,C*.
- Triples: *\A,B,C\: 1* (T1 only). < 2. $L_3^{(1)}$ = ∅.

Chunk 2.

- Singletons: *A: 3, B: 3, C: 3, D: 3*. All ≥ 2. $L_1^{(2)}$ = *\A, B, C, D*.
- Pairs: *\A,B\: 2* (T5, T6), *\A,C\: 2* (T5, T8), *\A,D\: 2* (T6, T8), *\B,C\: 2* (T5, T7), *\B,D\: 2* (T6, T7), *\C,D\: 2* (T7, T8). All ≥ 2. $L_2^{(2)}$ = all 6 pairs.
- Triples: *\A,B,C\: 1* (T5), *\A,B,D\: 1* (T6), *\A,C,D\: 1* (T8), *\B,C,D\: 1* (T7). All < 2. $L_3^{(2)}$ = ∅.

Chunk 3.

- Singletons: *A: 3* (T9, T11, T12), *B: 3* (T9, T10, T11), *C: 3* (T9, T10, T12), *D: 1* (T9). *D* < 2 — drop. $L_1^{(3)}$ = *\A, B, C*.
- Pairs: *\A,B\: 2* (T9, T11), *\A,C\: 2* (T9, T12), *\B,C\: 2* (T9, T10). All ≥ 2. $L_2^{(3)}$ = *\\A,B\, \A,C\, \B,C*.
- Triples: *\A,B,C\: 1* (T9). < 2. $L_3^{(3)}$ = ∅.

Local frequent union (= candidate set *C*):

- Singletons: *\A, B, C, D* (all four — *D* from chunk 2 only).
- Pairs: *\A,B\, \A,C\, \A,D\, \B,C\, \B,D\, \C,D* (all 6).
- Triples: none.

Pass 2 — count *C* over all 12 baskets globally.

Itemset	Baskets	σ (global)	≥ 4?
\A	T1, T2, T3, T5, T6, T8, T9, T11, T12	9	✓

Itemset	Baskets	σ (global)	≥ 4?
\B\	T1, T2, T4, T5, T6, T7, T9, T10, T11	9	✓
\C\	T1, T3, T4, T5, T7, T8, T9, T10, T12	9	✓
\D\	T6, T7, T8, T9	4	✓
\A,B\	T1, T2, T5, T6, T9, T11	6	✓
\A,C\	T1, T3, T5, T8, T9, T12	6	✓
\A,D\	T6, T8, T9	3	✗
\B,C\	T1, T4, T5, T7, T9, T10	6	✓
\B,D\	T6, T7, T9	3	✗
\C,D\	T7, T8, T9	3	✗

Global frequent itemsets (true output of SON):

- L_1 = \A, B, C, D\
- L_2 = \\A, B\, \A, C\, \B, C\\

False positives in action.

\A, D\, \B, D\, \C, D\ were local-frequent in chunk 2 but global-infrequent — pass 2 correctly weeds them out. SON guarantees no false negatives but does generate false positives.

EXAM TRAP — SON local threshold is reduced.

If you used global $s = 4$ in each chunk, every chunk would output essentially nothing (no itemset has count ≥ 4 inside a 4-basket chunk). You would then miss everything. Always reduce: local threshold = $s \cdot p_i$.

Worked Example 8 — Downward closure verification

Problem. Given that \A, B, C\ is a frequent 3-itemset (i.e. in L_3), list every itemset that downward closure forces to also be frequent. Then suppose someone claims that \A, B, D\ is also frequent — what additional itemsets must be frequent?

Part 1. Subsets of \A, B, C\ (excluding empty set):

Size	Subsets	Forced frequent?
1	\A\, \B\, \C\	✓ all three must be in L_1
2	\A,B\, \A,C\, \B,C\	✓ all three must be in L_2
3	\A,B,C\	✓ given

Total: 7 frequent itemsets (exactly $2^3 - 1$).

Part 2. \A, B, D\ frequent forces:

Size	Subsets	Already known frequent?
1	\A\, \B\, \D\	\A\, \B\ yes; \D\ NEW
2	\A,B\, \A,D\, \B,D\	\A,B\ yes; \A,D\, \B,D\ NEW
3	\A,B,D\	given

So 4 new itemsets: \D\, \A,D\, \B,D\, \A,B,D\.

Combined frequent inventory. L_1 = \A, B, C, D\, L_2 = \\A,B\, \A,C\, \B,C\, \A,D\, \B,D\\, L_3 = \\A,B,C\, \A,B,D\\.

Question — does L_3 imply L_4?

No. \A,B,C,D\ requires all four 3-subsets to be frequent: \A,B,C\ ✓, \A,B,D\ ✓, \A,C,D\ ?, \B,C,D\ ?. Without info on the last two, we cannot conclude \A,B,C,D\ is frequent. Downward closure works downward only — never upward.

The Apriori test in disguise.

This exact reasoning is what the prune step performs: given C_4 = \\A,B,C,D\\, look up its 3-subsets in L_3. \A,C,D\ ∉ L_3 ⇒ DROP \A,B,C,D\ before counting. WE 8 is the conceptual half of WE 2's mechanical join+prune.

§13 — Practice Questions (15)

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time targets: ~ *7* min per numerical, ~ *3* min per conceptual, ~ *1* min per MCQ.

Q1 [Numerical · 8 marks] ** Given the 5 baskets below and threshold $s = 2$, find L_1, L_2, and L_3 (Apriori). List all candidate sets and clearly mark dropped candidates.

TID	Items
T1	a, b, c
T2	a, c
T3	a, b, d
T4	b, c, d
T5	a, b, c, d

Q2 [Numerical · 6 marks] ** Given $L_2 = \{\{1,2\}, \{1,3\}, \{2,3\}, \{2,4\}, \{3,4\}, \{3,5\}\}$. Generate C_3 via join + prune. Clearly state the join condition and prune verdict for each candidate.

Q3 [Numerical · 7 marks] ** Frequent itemset $L = \{a, b, c\}$ has $\sigma(L) = 5$. Other supports: $\sigma(\{a\}) = 10$, $\sigma(\{b\}) = 8$, $\sigma(\{c\}) = 6$, $\sigma(\{a,b\}) = 7$, $\sigma(\{a,c\}) = 5$, $\sigma(\{b,c\}) = 6$. Compute confidence for all 6 rules derivable from L. With min-conf $= 80\%$, list all strong rules.

Q4 [Numerical · 8 marks] ** Run PCY pass 1 on the 6 baskets below with hash $h(i, j) = (i + j) \bmod 5$ and $s = 2$. (a) Build the singleton counts and identify L_1. (b) Fill the bucket-count table. (c) Construct the bitmap. (d) List pass 2 candidate pairs.

TID	Items
T1	1, 2, 3
T2	1, 4
T3	2, 3, 5
T4	1, 2, 4
T5	1, 3, 5
T6	2, 4, 5

Q5 [Numerical · 6 marks] ** Given baskets {milk, bread, butter}, {milk, bread}, {milk, eggs, bread, butter}, {bread, eggs}, {milk, butter}: compute $\text{sup}, \text{conf}, \text{lift}$ for the rule $\{\text{milk}\} \to \{\text{bread}\}$ and decide whether the rule is interesting.

Q6 [Numerical · 8 marks] ** SON on 9 baskets, 3 chunks, global $s = 3$. Use the chunks:

- Chunk 1: $\{a,b\}$, $\{a,b,c\}$, $\{a,c\}$
- Chunk 2: $\{b,c\}$, $\{a,b,c\}$, $\{b,c,d\}$
- Chunk 3: $\{a,d\}$, $\{a,c,d\}$, $\{c,d\}$

Find local-frequent itemsets in each chunk, the candidate set C, and the global frequent itemsets after pass 2.

Q7 [Concept · 4 marks] ** State the downward-closure (anti-monotone) property in formal terms. Prove the contrapositive (any superset of an infrequent itemset is infrequent). Why is this property essential for Apriori to outperform brute-force enumeration?

Q8 [Concept · 4 marks] ** Compare and contrast Apriori, PCY, and SON on (a) number of database scans, (b) memory profile, (c) suitability for distributed computing, (d) typical use case.

Q9 [Concept · 3 marks] ** Why is pass 2 of Apriori the dominant cost? Explain the "tyranny of counting pairs" and how PCY's pass-1 hash table addresses it.

Q10 [Concept · 4 marks] ** Define support, confidence, and lift. Give one numerical example where two rules have the same confidence but different lifts. Why is lift a better association quality measure?

Q11 [Concept · 3 marks] ** State the SON Lemma and explain why it guarantees no false negatives. Why does SON still need pass 2?

Q12 [MCQ · 2 marks] ** In Apriori's join step, you join two itemsets in L_k when they: (A) share any items in common (B) share their last $k - 1$ items in lex order (C) share their first $k - 1$ items in lex order (D) have empty intersection

Q13 [MCQ · 2 marks] ** PCY's hash table identifies a frequent bucket. Which is TRUE? (A) Every pair in that bucket is frequent. (B) At most one pair in that bucket is frequent. (C) Some pair in that bucket *might* be frequent — pass 2 must verify. (D) Pass 2 can skip every pair in that bucket.

Q14 [True/False · 2 marks] ** "If $\text{conf}(X \to Y) = 90\%$, then $\text{conf}(Y \to X)$ is also $\geq 90\%$." Justify.

Q15 [MCQ · 2 marks] ** In SON with 4 chunks of equal size, global threshold $s = 20$, the local threshold should be: (A) 20 (B) 80 (C) 5 (D) 4

---

§14 — Full Worked Answers

A1. ** *Singleton counts:*

Item	Baskets	σ	≥ 2?
a	T1, T2, T3, T5	4	✓
b	T1, T3, T4, T5	4	✓
c	T1, T2, T4, T5	4	✓
d	T3, T4, T5	3	✓

L_1 = \a, b, c, d\.

C_2 *— all pairs in lex order:* \a,b\, \a,c\, \a,d\, \b,c\, \b,d\, \c,d\.

Pair counts:

Pair	Baskets	σ	≥ 2?
\a,b\	T1, T3, T5	3	✓
\a,c\	T1, T2, T5	3	✓
\a,d\	T3, T5	2	✓
\b,c\	T1, T4, T5	3	✓
\b,d\	T3, T4, T5	3	✓
\c,d\	T4, T5	2	✓

L_2 = all 6 pairs.

C_3 *via join + prune.* Join shares first 1 item:

- \a,b\, \a,c\, \a,d\ → triples \a,b,c\, \a,b,d\, \a,c,d\.
- \b,c\, \b,d\ → \b,c,d\.

All 4 candidates have all 2-subsets in L_2 → all survive prune.

Triple counts:

Triple	Baskets	σ	≥ 2?
\a,b,c\	T1, T5	2	✓

Triple	Baskets	σ	≥ 2?
\a,b,d\	T3, T5	2	✓
\a,c,d\	T5	1	✗
\b,c,d\	T4, T5	2	✓

L_3 = \\a,b,c\, \a,b,d\, \b,c,d\\.

C_4. Candidate \a,b,c,d\ from joining \a,b,c\ and \a,b,d\. Prune: 3-subsets are \a,b,c\ ∈ L_3, \a,b,d\ ∈ L_3, \a,c,d\ ∉ L_3, \b,c,d\ ∈ L_3. **DROP** because \a,c,d\ missing.

L_4 = ∅. Algorithm halts.

A2. ** Sort items in each itemset (already done). Join:

- \1,2\, \1,3\ share \1\ → \1,2,3\. ✓
- \2,3\, \2,4\ share \2\ → \2,3,4\. ✓
- \3,4\, \3,5\ share \3\ → \3,4,5\. ✓

Prune:

- \1,2,3\ — 2-subsets \1,2\, \1,3\, \2,3\ — all in L_2. ✓ KEEP.
- \2,3,4\ — 2-subsets \2,3\, \2,4\, \3,4\ — all in L_2. ✓ KEEP.
- \3,4,5\ — 2-subsets \3,4\, \3,5\, \4,5\ — \4,5\ ∉ L_2. ✗ DROP.

C_3 = \\1,2,3\, \2,3,4\\.

A3. **

Rule	Numerator σ	Denominator σ	Confidence	≥ 80\%?
\a,b\ → \c\	5	7	71.4%	✗
\a,c\ → \b\	5	5	100%	✓
\b,c\ → \a\	5	6	83.3%	✓
\a\ → \b,c\	5	10	50%	✗
\b\ → \a,c\	5	8	62.5%	✗
\c\ → \a,b\	5	6	83.3%	✓

Strong rules: \a,c\ → \b\, \b,c\ → \a\, \c\ → \a,b\.

A4. **

(a) **Singleton counts.** 1: 4 (T1,T2,T4,T5), 2: 4 (T1,T3,T4,T6), 3: 3 (T1,T3,T5), 4: 3 (T2,T4,T6), 5: 3 (T3,T5,T6). All ≥ 2. L_1 = \1, 2, 3, 4, 5\.

(b) **Pairs by basket.** T1: (1,2)(1,3)(2,3). T2: (1,4). T3: (2,3)(2,5)(3,5). T4: (1,2)(1,4)(2,4). T5: (1,3)(1,5)(3,5). T6: (2,4)(2,5)(4,5).

Hash *h(i,j) = (i+j) bmod 5*:

Pair	i+j	bucket
(1,2)	3	3

Pair	*i+j*	bucket
(1,3)	4	4
(1,4)	5	0
(1,5)	6	1
(2,3)	5	0
(2,4)	6	1
(2,5)	7	2
(3,5)	8	3
(4,5)	9	4

Pair tallies: (1,2):2 ; (1,3):2 ; (1,4):2 ; (1,5):1 ; (2,3):2 ; (2,4):2 ; (2,5):2 ; (3,5):2 ; (4,5):1.

Bucket counts:

- Bucket 0: (1,4):2 + (2,3):2 = **4** ✓
- Bucket 1: (1,5):1 + (2,4):2 = **3** ✓
- Bucket 2: (2,5):2 = **2** ✓
- Bucket 3: (1,2):2 + (3,5):2 = **4** ✓
- Bucket 4: (1,3):2 + (4,5):1 = **3** ✓

(c) **Bitmap = (1, 1, 1, 1, 1)** — every bucket frequent.

(d) **Pass 2 candidates.** Both items in L_1 (yes for all pairs since L_1 is everything) AND bucket frequent (yes for all). So **all 9 distinct pairs are candidates**. PCY did not prune anything in this case — happens when threshold and bucket count are tightly coupled. Pass 2 then counts each candidate: (1,2):2 ✓, (1,3):2 ✓, (1,4):2 ✓, (1,5):1 ✗, (2,3):2 ✓, (2,4):2 ✓, (2,5):2 ✓, (3,5):2 ✓, (4,5):1 ✗. L_2 = *1,2*\, *1,3*\, *1,4*\, *2,3*\, *2,4*\, *2,5*\, *3,5*\\.

A5. ** *N = 5*. *σ(milk) = 4* (T1,T2,T3,T5), *σ(bread) = 4* (T1,T2,T3,T4), *σ(\milk, bread\) = 3* (T1,T2,T3).

- Support: *3/5 = 0.60*
- Confidence: *3/4 = 0.75*
- Lift: *0.75 / (4/5) = 0.75 / 0.80 = 0.9375*

Lift < *1* — milk and bread are slightly *anti-correlated*. Although the rule has high support and confidence, the rule is not "interesting" in the lift sense — knowing milk slightly *decreases* the probability of bread compared to the marginal.

A6. ** Local threshold = *3 · (1/3) = 1*, so any itemset present in any chunk is local-frequent.

Chunk 1 frequent: *\a\:3, \b\:2, \c\:2, \a,b\:2, \a,c\:2, \b,c\:1, \a,b,c\:1* (every itemset that appears).

Chunk 2 frequent: *\a\:1, \b\:3, \c\:3, \d\:1, \a,b\:1, \a,c\:1, \b,c\:3, \b,d\:1, \c,d\:1, \a,b,c\:1, \b,c,d\:1*.

Chunk 3 frequent: *\a\:2, \c\:2, \d\:3, \a,c\:1, \a,d\:2, \c,d\:2, \a,c,d\:1*.

Candidate set C = union of all = every itemset listed above.

Pass 2 — count globally over 9 baskets.

Singletons: $\sigma(a)$ = baskets B1+B2 from C1, B5 from C2 (assuming basket 5 = {a,b,c}), B7,B8 from C3 = let's just count: from C1: B1{a,b}, B2{a,b,c}, B3{a,c} → 3. From C2: B5{a,b,c} → 1. From C3: B7{a,d}, B8{a,c,d} → 2. Total $\sigma(a) = 6$. ≥ 3 ✓.

$\sigma(b)$: C1: B1, B2 → 2; C2: B4{b,c}, B5{a,b,c}, B6{b,c,d} → 3; C3: 0. Total 5. ≥ 3 ✓.

$\sigma(c)$: C1: B2, B3 → 2; C2: B4, B5, B6 → 3; C3: B8, B9{c,d} → 2. Total 7. ≥ 3 ✓.

$\sigma(d)$: C1: 0; C2: B6 → 1; C3: B7, B8, B9 → 3. Total 4. ≥ 3 ✓.

Pairs: $\sigma(\{a,b\})$ = B1, B2 (C1), B5 (C2) = 3. ≥ 3 ✓. $\sigma(\{a,c\})$ = B2, B3 (C1), B5 (C2), B8 (C3) = 4. ≥ 3 ✓. $\sigma(\{a,d\})$ = B7, B8 (C3) = 2. ✗. $\sigma(\{b,c\})$ = B2 (C1), B4, B5, B6 (C2) = 4. ≥ 3 ✓. $\sigma(\{b,d\})$ = B6 (C2) = 1. ✗. $\sigma(\{c,d\})$ = B6 (C2), B8, B9 (C3) = 3. ≥ 3 ✓.

Triples: $\sigma(\{a,b,c\})$ = B2 (C1), B5 (C2) = 2. ✗. $\sigma(\{b,c,d\})$ = B6 (C2) = 1. ✗. $\sigma(\{a,c,d\})$ = B8 (C3) = 1. ✗.

Global frequent itemsets: $L_1 = \{a, b, c, d\}$, $L_2 = \{\{a,b\}, \{a,c\}, \{b,c\}, \{c,d\}\}$, $L_3 = \emptyset$.

A7. ** Statement: "If I is frequent then every $J \subseteq I$ with $J \neq \emptyset$ is also frequent." Contrapositive: "If J is infrequent then every superset $I \supseteq J$ is also infrequent." Proof of contrapositive: if $\sigma(J) < s$ and $I \supseteq J$, every basket containing I also contains J, so $\sigma(I) \leq \sigma(J) < s$. Thus I is infrequent. *square*

The property is essential because brute-force enumeration of 2^M itemsets is intractable for M in the thousands. Apriori prunes the search tree by never generating any candidate that has an infrequent subset — typical real-world prune rates exceed 99%. Without it, Apriori would need to count exponentially many itemsets.

A8. **

Aspect	Apriori	PCY	SON				
DB scans	k (one per level)	k (same)	2 (regardless of k)				
Memory	$O(\	C_2\	)$ pass 2 dominates	$O(\	C_2\	)$ reduced by bucket bitmap	local memory per chunk + candidate set C
Distributed-friendly	not directly	not directly	yes — embarrassingly parallel pass 1				
Typical use	small in-memory data	one-machine data with tight RAM	data too big for one machine, MapReduce				

A9. ** Pass 1 produces $|L_1| = m$ frequent singletons. Pass 2 must count $binomm2 = m(m-1)/2$ candidate pairs. Each pair needs a counter — typically 4 bytes. With $m = 10^4$, this is ≈ 200 *MB* of pair counters. With $m = 10^5$, ≈ 20 *GB* — exceeding most single-machine RAM. PCY uses pass 1's spare memory (singleton counters take only $O(m)$, leaving $O(m^2)$ free) to maintain a hash table of pair counts. After pass 1, infrequent buckets cannot contain any frequent pair — so pass 2 only counts pairs whose hash bucket exceeds the threshold AND whose items are individually frequent. Typical 5–10× reduction in pass-2 memory.

A10. **

- Support of rule $X \to Y$: $\sigma(X \cup Y) / N$. Fraction of baskets where rule is observed.
- Confidence: $\sigma(X \cup Y) / \sigma(X)$. Conditional probability $P(Y \mid X)$.
- Lift: $conf(X \to Y) / sup(Y) = N \cdot \sigma(X \cup Y) / (\sigma(X) \cdot \sigma(Y))$. Ratio to independence baseline.

Example: 1000 baskets. Rule R1: $\sigma(X)=500$, $\sigma(Y)=500$, $\sigma(X \cup Y)=300$. conf = 60%, lift = 0.60/0.50 = 1.2. Rule R2: $\sigma(X)=500$, $\sigma(Y)=900$, $\sigma(X \cup Y)=300$. conf = 60%, lift = 0.60/0.90 ≈ 0.67. Same confidence but R1 is positively associated, R2 negatively. Lift is a better measure because it normalises for the marginal probability of Y — high confidence to a popular consequent isn't surprising.

A11. ** SON Lemma: if itemset I is frequent in entire D (count ≥ s), then I must be locally frequent in at least one chunk with reduced threshold $p_i \cdot s$. Reason: if I were below local threshold in EVERY chunk, summing $|D_i| \cdot local\ sup(I) < |D_i| \cdot p_i \cdot s$ across chunks gives total $< |D| \cdot s$, contradicting global frequency.

This guarantees no false negative: anything globally frequent is in the union of local-frequent sets. Pass 2 is still needed because false positives are possible — an itemset can be local-frequent in one chunk but globally infrequent (it just barely cleared the local threshold but is rare elsewhere).

A12. (C)**. The first $k - 1$ items in lex order must match. Last items must differ ($p_k < q_k$). This avoids duplicates and produces each $(k+1)$-candidate exactly once.

A13. (C)**. Bucket frequency is a *necessary* condition (any frequent pair must hash to a frequent bucket since the bucket's count includes that pair). It is not *sufficient* — multiple pairs hash to one bucket and share the bucket's count. Pass 2 must verify by counting individual pairs.

A14. False.** Confidence is asymmetric. Counterexample: 100 baskets, every basket contains "car"; 5 baskets contain "Ferrari". Then $conf(Ferrari \to car) = 5/5 = 100\%$ but $conf(car \to Ferrari) = 5/100 = 5\%$. The rare item appearing implies the common item, but not vice versa.

A15. (C) 5.** Local threshold = global threshold × chunk fraction = $20 \times (1/4) = 5$.

---

§15 — Ending Key Notes (Revision Cards)

Term	Quick-fact				
Basket / transaction	Subset of items. Database D is a list of baskets.				

Term	Quick-fact				
Item universe *I*	All distinct items. $	\mathcal{I}	= M$.		
Support count *σ(I)*	# baskets containing *I*. Integer.				
Support fraction *sup(I)*	*σ(I)/N*. Probability. Between 0 and 1.				
Threshold *s*	Either count or fraction — read carefully.				
L_k	Frequent *k*-itemsets.				
C_k	Candidate *k*-itemsets (before counting).				
Downward closure	Subset of frequent is frequent. Drives all pruning.				
Apriori principle	Drop candidate if any subset infrequent.				
Join step	Match first *k-1* items in lex order; last items differ.				
Prune step	Drop candidate if any *k*-subset not in L_k.				
Confidence	*σ(X ∪ Y) / σ(X)*. Asymmetric.				
Lift	conf / sup(Y). Lift > 1: positive assoc. Lift = 1: independence.				
Pass-2 bottleneck	$\binom{	L_1	}{2} *pair counters — quadratic in*	L_1	$.
PCY pass 1	Hash every basket's pairs into bucket counter table.				
Bitmap	1 bit per bucket: 1 if bucket count ≥ *s*.				
PCY pass 2 candidate	Both items in L_1 AND bucket frequent.				
Multistage	Multiple sequential hash passes — each prunes further.				
Multihash	Multiple hash tables in parallel during pass 1.				
SON Lemma	Globally frequent ⟹ locally frequent in some chunk.				

Term	Quick-fact				
SON local threshold	$p_i \cdot s$ where p_i = chunk fraction.				
FP-tree	Compact trie of sorted baskets. Two scans.				
Conditional pattern base	Sub-database of patterns ending at item *a*.				
Toivonen's algo	Sample + negative border → no false negatives w.h.p.				
Mistake — count vs fraction	Always check the threshold's units.				
Mistake — join condition	Match first *k-1* items, NOT just any pair.				
Mistake — skip prune	Always check all *k*-subsets are in L_k.				
Mistake — PCY bucket reading	Frequent bucket ≠ frequent pair. Pass 2 must verify.				
Mistake — confidence asymmetry	$conf(X \to Y) \neq conf(Y \to X)$.				
Mistake — SON local threshold	Reduce by p_i, don't use global *s*.				

§16 — Formula & Algorithm Reference

Concept	Formula	When to use				
Support count	$\sigma(I) =	\{T \in D : I \subseteq T\}	$	Defining frequency.		
Support fraction	$sup(I) = \sigma(I)/N$	Probability interpretation.				
Frequent itemset	$\sigma(I) \geq s$	Membership in L_k.				
Confidence	$conf(X \to Y) = \sigma(X \cup Y) / \sigma(X)$	Conditional probability $P(Y	X)$.			

Concept	Formula	When to use				
Lift	*lift(X → Y) = N · σ(X ∪ Y) / (σ(X) · σ(Y))*	Independence baseline check.				
Conviction	*(1 - sup(Y)) / (1 - conf(X → Y))*	Direction-sensitive measure.				
Apriori join	$p, q \in L_k$: match first *k-1* items, $p_k < q_k \to p \cup q \in C_{k+1}$	Generating candidates.				
Apriori prune	Drop $c \in C_{k+1}$ if any *k*-subset $\notin L_k$	Avoiding wasted counts.				
PCY hash	*h(i, j)* = hash function, *bmod B*	Pass-1 bucket table.				
PCY candidate	$i, j \in L_1$ AND *bucket[h(i,j)] ≥ s*	Pass-2 filter.				
SON local threshold	$p_i \cdot s$ where $p_i =	D_i	/	D	$	Each chunk's mining threshold.
Number of rules from *L*	$2^{	L	} - 2$	Total non-trivial rules from a frequent itemset.		

Algorithmic complexity:

Algorithm	Passes	Pass-2 memory	Strength		
Apriori	*k*	$O(\	C_2\	)$ pair counters	Simple, well-understood
PCY	*k*	$O(\	C_2\	)$ but pruned by bitmap	Saves pass-2 RAM 5–10×
Multistage	*k+1*	further reduced	Pass-2 RAM critical		
Multihash	*k*	reduced via two hashes	Moderate RAM		
SON	2	$O(\	C\	)$ candidate set	Distributed, MapReduce

Algorithm	Passes	Pass-2 memory	Strength		
FP-Growth	2	FP-tree	Dense data, no candidates		

Connections to other weeks:

- **W04 — MapReduce:** SON is the canonical MapReduce ARM algorithm. The map/reduce decomposition in §9 mirrors W04's job-level templates.
- **W05–W06 — Frequent Items Streams:** when D is a never-ending stream, we replace L_1 counting with the Misra–Gries / Lossy-Counting algorithms; everything else (PCY, SON) ports.
- **W07 — Recommendation Systems:** market-basket co-occurrence is a primitive collaborative-filtering signal. Item-item similarity uses the same σ tables.
- **W10–W11 — Clustering:** clusters of items can be derived from frequent itemsets (Wenskowski–Zaki style).
- **MMDS Chapter 6:** the canonical reference. Sections 6.1 (market-basket), 6.2 (Apriori), 6.3 (PCY/multistage/multihash), 6.4 (SON/Toivonen). Read 6.2 and 6.3 ten times before the exam.

End of W02-03 Distributed Association Rule Mining exam-prep document.